

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent Application Publication

20250279790

Kind Code

A1

Publication Date

September 04, 2025

Inventor(s)

Slobodskoy; Vitaly Vitalyevich et al.

Method and Device for Converting Representations of Values in Different Systems

Abstract

Embodiments of the present application provide a method and a device for converting representations of values between RNS and CRNS. The present application can be used in any product to which RNS is applied. A compact RNS (called CRNS) is proposed by the present application, and a bijection between RNS representations and CRNS representations is created. The bijection maps between $(n+1)$ -bit RNS representations and n -bit CRNS representations by using a logic function. The bijection maps only $2^{\sup n}$ values corresponding to an n -bit BNS value in a dynamic range $[0, 2^{\sup n}-1]$.

Inventors: Slobodskoy; Vitaly Vitalyevich (Nizhny Novgorod, RU), Plesner; Erik Konstatin (Nizhny Novgorod, RU), Ignatov; Oleg Vasilyevich (Nizhny Novgorod, RU), Kurmaev; Roman Alekseevich (Moscow, RU), Ryabchikova; Valeria Georgievna (Nizhny Novgorod, RU), Martirosyan; Elizaveta Robertovna (Nizhny Novgorod, RU)

Applicant: Huawei Technologies Co., Ltd. (Shenzhen, CN)

Family ID: 84799854

Appl. No.: 19/208297

Filed: May 14, 2025

Related U.S. Application Data

parent WO continuation PCT/RU2022/000337 20221115 PENDING child US 19208297

Publication Classification

Int. Cl.: H03M7/18 (20060101)

Background/Summary

CROSS-REFERENCE TO RELATED APPLICATIONS [0001] This application is a continuation of International Application No. PCT/RU2022/000337, filed on Nov. 15, 2022, the disclosure of which is hereby incorporated by reference in its entirety.

TECHNICAL FIELD

[0002] Embodiments of the present invention relate to the field of residual number system (RNS), and more specifically, to a method and device for converting representations of values in different systems. cl BACKGROUND

[0003] A residual number system (RNS) is a number representation system, which can obtain a set of remainders of a value by performing remainder operations on a set of moduli. The dynamic range $[0; M-1]$ is the number of values which have a unique representation in the RNS. M is the product of all the moduli of the RNS. There is no weight and propagation between channels corresponding to each remainder. Using the RNS, an operation of a value with a long bit width can be converted into an operation of a set of remainders with a shorter bit width of the value. Hardware resources and delay required for a single operation are smaller compared to a binary number system (BNS), and parallelism of the operation is improved. In addition, the hardware resources can also be saved. The RNS is widely used after several years of development because of these properties.

[0004] It is known that a representation of a value in the BNS is different from a representation of the value in the RNS, and the representation in the RNS is related to the moduli of the RNS. Consider an n -bit BNS, and the BNS has a dynamic range of $2^{\sup.n}$, that is, the n -bit BNS can represent $2^{\sup.n}$ values in total, namely the values $[0; 2^{\sup.n}-1]$. It requires an RNS of at least $n+1$ bits to represent a BNS with a dynamic range of $2^{\sup.n}$. This difference between the representation of the value in the BNS and the representation of the value in the RNS brings many disadvantages to the application of the RNS, for an example, a conversion between the BNS representation and the RNS representation will lead to excessive storage overhead and memory bandwidth overhead

SUMMARY

[0005] Embodiments of the present application provide a method for converting representations of values in different systems, which can reduce storage consumption and memory bandwidth.

[0006] According to a first aspect, provided is a method for converting representations of values in different systems, including: [0007] receiving a first representation of a first value, where the first representation is a representation of the first value in a residual number system (RNS), and the first representation contains $n+1$ bits; [0008] converting the first representation to a second representation according to a truncated bit, where the second representation is a representation of the first value in a compact residual number system (CRNS), and the second representation contains n bits, [0009] where the truncated bit is any one of the $n+1$ bits, and if the truncated bit of the $n+1$ bits is equal to zero, the second representation is a copy of remaining bits of the $n+1$ bits contained in the first representation that exclude the truncated bit, and if the truncated bit of the $n+1$ bits is equal to one, the second representation is a copy of recoded remaining n bits of the $n+1$ bits contained in the first representation that exclude the truncated bit; [0010] where the first value is one of $2^{\sup.n}$ values, the $2^{\sup.n}$ values can be represented by an n -bit BNS, and a value range of the $2^{\sup.n}$ values is $[0; 2^{\sup.n}-1]$, n is an integer.

[0011] According to the present application, a compact RNS (called CRNS in the present application) is proposed, and the $n+1$ -bit RNS representations and n -bit CRNS representations can be converted to each other. By storing n -bit CRNS representations, compared to storing $n+1$ -bit RNS representations, the storage consumption is reduced.

[0012] In addition, the inconvenience of converting between $n+1$ -bit RNS representations and n -bit BNS values can be resolved, and the power consumption is reduced.

[0013] According to a second aspect, provided is a method for converting representations of values in different systems, including: [0014] receiving a second representation of a first value, where the second representation is a representation of the first value in a compact residual number system (CRNS), and the second representation contains n bits; [0015] determining whether the second representation is identical in the CRNS and an RNS; [0016] converting the second representation to a first representation, where the first representation is a representation of the first value in the RNS, and the first representation contains $n+1$ bits; [0017] where if the second representation is identical in the CRNS and the RNS, the first representation is a copy of the n bits of the second representation and a truncated bit added by zero, and if the second representation is different in the CRNS and the RNS, the first representation is a copy of recoded n bits of the second representation and the truncated bit added by one; [0018] where the first value is one of $2^{\text{sup}.n}$ values, the $2^{\text{sup}.n}$ values can be represented by an n -bit BNS, and a value range of the $2^{\text{sup}.n}$ values is $[0; 2^{\text{sup}.n}-1]$, n is an integer.

[0019] The technique effect of the method of the second aspect could refer to that of the first aspect, and is not repeated here.

[0020] In an embodiment of the first aspect or the second aspect, where the $2^{\text{sup}.n}$ values correspond to $2^{\text{sup}.n}$ first representations in the RNS, the $2^{\text{sup}.n}$ values correspond to $2^{\text{sup}.n}$ second representations in the CRNS, and the $2^{\text{sup}.n}$ first representations and the $2^{\text{sup}.n}$ second representations are in a bijection mapping relationship.

[0021] According to this embodiment, a bijection between RNS representations and CRNS representations is created. Specifically, the bijection maps between $(n+1)$ -bit RNS representations and n -bit CRNS representations by using a logic function. The bijection maps only $2^{\text{sup}.n}$ values corresponding to an n -bit BNS value in a range $[0, 2^{\text{sup}.n}-1]$. The BNS values larger than or equal to $2^{\text{sup}.n}$ are disregarded, and these BNS values will not be used and are not needed. The bijection allow using only n bits to represent all $2^{\text{sup}.n}$ BNS values of an n -bit BNS. Comparing with storing $(n+1)$ -bit RNS representations of values, the storage consumption is reduced by storing n -bit CRNS representations.

[0022] Consider an n -bit BNS, and the BNS has a dynamic range of $2^{\text{sup}.n}$. It has been proved by the present application that, an RNS representation of a BNS value in the dynamic range $[0; 2^{\text{sup}.n}-1]$ will always require at least $n+1$ bits. Conversion between n -bit BNS representations and $(n+1)$ -bit RNS representations leads to memory storage consumption, for example, an 8-bit BNS representation becomes a 9-bit RNS representation, and actually takes up 2 pcs of 8-bit positions in memory. It requires more memory space to store RNS representations than BNS representations. The proposed solution by the present application allows to use n -bit CRNS representation to map an n -bit BNS representation, hence the memory consumption is reduced.

[0023] In an embodiment of the first aspect or the second aspect, where the $2^{\text{sup}.n}$ first representations include f first representations of a first type and $(2^{\text{sup}.n}-f)$ first representations of a second type, the truncated bit of the first representation of the first type is equal to one, and the truncated bit of the first representation of the second type is equal to zero; [0024] where the f first representations of the first type correspond to f second representations, and each of the f second representations is a copy of recoded remaining n bits of the corresponding first representation of the first type that exclude the truncated bit; [0025] where the $(2^{\text{sup}.n}-f)$ first representations of the second type correspond to $(2^{\text{sup}.n}-f)$ second representations, and each of the $(2^{\text{sup}.n}-f)$ second representations is a copy of the remaining n bits of the corresponding first representation of the

second type that exclude the truncated bit.

[0026] In an embodiment of the first aspect or the second aspect, the bijection mapping relationship between the f first representations of the first type and the f second representations is adjustable.

[0027] According to this embodiment, the actual CRNS values do not matter. The only thing which matters, is that the mapping is a bijection. In order to do logic optimization, only the recoded RNS representations are permuted, and the RNS representations which are “copied” are not permuted. By doing so, the hardware complexity of the conversions between RNS and CRNS is reduced.

[0028] In an embodiment of the first aspect or the second aspect, where the $n+1$ bits of the first representation contain k parts, the k parts are residuals corresponding to the k moduli of the RNS, and the k parts are arranged in one of the following orders: [0029] in an ascending order according to corresponding values of the k moduli; [0030] in a descending order according to corresponding values of the k moduli; or [0031] in a random order.

[0032] In an embodiment of the first aspect or the second aspect, the $n+1$ bits of the k parts are interleaved, and the truncated bit is one of the interleaved $n+1$ bits.

[0033] In an embodiment of the first aspect or the second aspect, where the k parts contain a first part, and the first part contains $p_{\text{sub},i}$ bits, the truncated bit is any one of the $p_{\text{sub},i}$ bits, $p_{\text{sub},i} = \lfloor \log_2(m_i - 1) \rfloor$, $m_{\text{sub},i}$ is a modulo of the k moduli corresponding to the first part, and the first part is any one of the k parts, $1 \leq i \leq k$, i and k are integers.

[0034] In an embodiment of the first aspect or the second aspect, where the truncated bit is a most significant bit (MSB) of the $p_{\text{sub},i}$ bits.

[0035] According to this embodiment, use of the MSB of the truncated modulo as the truncated bit usually leads to fewer RNS representations with the $RNS_{\text{sub},j} = 1$, that is, the number of the RNS representations needs to be recoded is reduced. The storage area, power consumption and latency for conversion are reduced.

[0036] In an embodiment of the first aspect or the second aspect, where the $m_{\text{sub},i}$ is being the form $m_{\text{sub},i} = 2^{\text{sup},q_{\text{sub},i}+1}$, $1 \leq i \leq k$, i is an integer and $q_{\text{sub},i}$ is an integer. Note that $q_{\text{sub},i}$ might not exist for particular $m_{\text{sub},i}$ since not every modulo might be in form of $2^{\text{sup},q_{\text{sub},i}+1}$. That is to say, for the m moduli of the moduli-set, $q_{\text{sub},i}$ exist just for the moduli which are being the form of $2^{\text{sup},q_{\text{sub},i}+1}$. For example, if the moduli-set is $\{m_{\text{sub},1}, m_{\text{sub},2}\} = \{5_{\text{sub},10}, 13_{\text{sub},10}\}$, the modulo $m_{\text{sub},1}$ is being the form of $m_{\text{sub},i} = 2^{\text{sup},q_{\text{sub},i}+1}$, specifically, $m_{\text{sub},1} = 2^{\text{sup},q_{\text{sub},1}+1} = 2^{\text{sup},2+1}$ and $q_{\text{sub},1}$ is equal to 2. $q_{\text{sub},2}$ is not exist for the modulo $m_{\text{sub},2}$.

[0037] According to this embodiment, the use of the truncated modulo of the form $m_{\text{sub},i} = 2^{\text{sup},q_{\text{sub},i}+1}$ and the use of the MSB of the said modulo $m_{\text{sub},i}$ as the truncated bit lead to fewer RNS representations that needs to be recoded, hence the storage area, power consumption and latency for conversion are reduced. Compared with the use of the MSB of the truncated modulo as the truncated bit, this embodiment leads to fewer recoded RNS representations, and storage area, power consumption and latency for conversion are further reduced.

[0038] In an embodiment of the first aspect or the second aspect, where the $m_{\text{sub},i}$ is the modulo that has the largest value of the k moduli.

[0039] According to this embodiment, the use of the MSB of the truncated modulo of the form $m_{\text{sub},k} = 2^{\text{sup},q_{\text{sub},k}+1}$ which is the highest value modulo in a moduli-set leads to fewest possible recoded values in the bijection, and the advantages, such as storage area, power consumption, latency for conversion and so on, are maximized.

[0040] According to a third aspect, a chip (or a chip system) is provided. The chip has a function of implementing the method in the first aspect or in the second aspect, and any embodiments of the first aspect or the second aspect. The function may be implemented by using hardware structures. Alternatively, the chip or chip system includes an interface and a plurality of circuits.

[0041] According to a fourth aspect, a device is provided, including a chip or a chip system of the third aspect.

Description

BRIEF DESCRIPTION OF DRAWINGS

[0042] One or more embodiments are exemplarily described by corresponding accompanying drawings, and these exemplary illustrations and accompanying drawings constitute no limitation on the embodiments. Elements with the same reference numerals in the accompanying drawings are illustrated as similar elements, and the drawings are not limiting to scale, in which:

[0043] FIG. 1 is a schematic diagram of a method for converting first representations of values to second representations.

[0044] FIG. 2 is an example of a number of recoded RNS values in an 8-bit CRNS.

[0045] FIG. 3 shows an example of a trend graph of f for n=8.

[0046] FIG. 4 is a schematic diagram of a method for converting second representations of values to first representations.

[0047] FIG. 5 is a schematic diagram of an embodiment of converting from RNS to CRNS.

[0048] FIG. 6 is a schematic diagram of an embodiment of converting from RNS to CRNS.

[0049] FIG. 7 is a schematic diagram of an embodiment of converting from CRNS to RNS.

[0050] FIG. 8 is a schematic diagram of an embodiment of converting from CRNS to RNS.

[0051] FIG. 9 is a schematic diagram of an embodiment of converting from CRNS to RNS.

[0052] FIG. 10 is a schematic diagram of an embodiment of converting from CRNS to RNS.

[0053] FIG. 11 is a schematic block diagram of a chip or a chip system according to an embodiment of the present application.

DESCRIPTION OF EMBODIMENTS

[0054] In order to understand features and technical contents of embodiments of the present disclosure in detail, implementations of the embodiments of the present disclosure will be described in detail below with reference to the accompanying drawings, and the attached drawings are only for reference and illustration purposes, and are not intended to limit the embodiments of the present disclosure. In the following technical descriptions, for ease of explanation, numerous details are set forth to provide a thorough understanding of the disclosed embodiments. One or more embodiments, however, may be practiced without these details. In other cases, well-known structures and apparatuses may be shown simplified in order to simplify the drawings.

[0055] Related technologies and concepts are introduced here firstly in order to better understand the technique solution proposed by the present application.

1. Residue Number System (RNS)

[0056] A RNS is defined by a set of k integers $\{m_{\text{sub}.1}, m_{\text{sub}.2}, \dots, m_{\text{sub}.k}\}$ called a moduli-set. Any two integers in the moduli-set must be pairwise coprime, i.e. $\text{GCD}(m_{\text{sub}.i}, m_{\text{sub}.j})=1$, $i, j \in [1, k]$, $i \neq j$. Let M be a product of all $m_{\text{sub}.i}$: $M = \prod_{i=1}^k m_{\text{sub}.i}$, then an integer x in a range of $[0, M-1]$ is uniquely represented in the RNS by a set of its residues $x_{\text{fwdarw}.1}, x_{\text{fwdarw}.2}, \dots, x_{\text{fwdarw}.k}$ under Euclidean division by the moduli-set; that is, $r_{\text{sub}.i} = x \bmod m_{\text{sub}.i}$ and $0 \leq r_{\text{sub}.i} < m_{\text{sub}.i}$ for each i.

[0057] According to Chinese Remainder Theorem (CRT), x can be reconstructed from RNS as:

[00001]

$$x = \sum_{i=1}^k r_i M_i^{-1} \prod_{j \neq i} M_j \pmod{M}, M_i = \frac{M}{m_i}, M_i M_i^{-1} \pmod{m_i} = 1$$

[0058] The RNS has many advantages, such as independent (parallel) addition, subtraction and multiplication operations between corresponding remainders:

[00002]

$$a + b = \{ \sum_{i=1}^k a_i + b_i \pmod{m_i}, \sum_{i=2}^k a_i + b_i \pmod{m_2}, \dots, \sum_{i=k}^k a_i + b_i \pmod{m_k} \}$$

$$a - b = \{ \text{.Math. } a_1 - b_1 \text{ .Math. }_{m_1}, \text{ .Math. } a_2 - b_2 \text{ .Math. }_{m_2}, \text{ .Math. } a_k - b_k \text{ .Math. }_{m_k} \}$$

$$a * b = \{ \text{.Math. } a_1 * b_1 \text{ .Math. }_{m_1}, \text{ .Math. } a_2 * b_2 \text{ .Math. }_{m_2}, \text{ .Math. } a_k * b_k \text{ .Math. }_{m_k} \}$$

[0059] Lack of carry propagation is a main interesting aspect of the RNS, because it reduces complexity, reduces power consumption and speed up the calculations compared with usual binary number system (BNS) representation.

[0060] The RNS also has limitations, for example, non-modular operations (comparison, scaling, etc.) are generally complex in RNS, transformation is slow between positioning and modular representations, and an overflow detection is difficult.

[0061] In the present application, it is important in the rest of the document to understand differences and to distinguish between a modulo $m_{\text{sub}.i}$ and a residual $r_{\text{sub}.i}$ under the modulo $m_{\text{sub}.i}$, that is, $r_{\text{sub}.i} = |x|_{\text{sub}.m_{\text{sub}.i}}$, where x is a BNS value being represented.

[0062] In addition, it is important to distinguish between “representation” and “value”. In BNS, a representation of a value x is a binary representation of x . The value x in the RNS and BNS is the same, but representations of the value x in the RNS and BNS are different.

[0063] For example, an integer $99_{\text{sub}.10}$ has a representation $1100011_{\text{sub}.2}$ in the BNS. In the RNS, the representation of the value x depends on the moduli-set of the RNS.

[0064] For example, for the RNS, in a given moduli-set $m = \{3_{\text{sub}.10}, 5_{\text{sub}.10}, 7_{\text{sub}.10}\}$ of $M = 105_{\text{sub}.10}$, the value $99_{\text{sub}.10}$ gets a representation $r = \{0, 4_{\text{sub}.10}, 1_{\text{sub}.10}\}$ under the said moduli-set. For the said RNS, the binary form of residuals becomes $\{00_{\text{sub}.2}, 100_{\text{sub}.2}, 001_{\text{sub}.2}\}$.

[0065] It can be shown from the above example, residuals $r_{\text{sub}.1} = 0_{\text{sub}.10} = 00_{\text{sub}.2}$ occupies 2 bits, $r_{\text{sub}.2} = 4_{\text{sub}.10} = 100_{\text{sub}.2}$ occupies 3 bits, and $r_{\text{sub}.3} = 1_{\text{sub}.10} = 001_{\text{sub}.2}$ occupies 3 bits. In total $2_{\text{sub}.10} + 3_{\text{sub}.10} + 3_{\text{sub}.10} = 8_{\text{sub}.10}$ bits for the RNS representation, the residuals (that is, $r_{\text{sub}.1}$, $r_{\text{sub}.2}$ and $r_{\text{sub}.3}$) are encoded into an 8-bit “RNS word” as shown below in Table 1.

TABLE-US-00001 TABLE 1 RNS bit 7 6 5 4 3 2 1 0 Residual bit $r_{\text{sub}.3.2}$ $r_{\text{sub}.3.1}$ $r_{\text{sub}.3.0}$ $r_{\text{sub}.2.2}$ $r_{\text{sub}.2.1}$ $r_{\text{sub}.2.0}$ $r_{\text{sub}.1.1}$ $r_{\text{sub}.1.0}$ Representation 0 0 1 1 0 0 0 0 of “ $99_{\text{sub}.10}$ ” In Table 1, “ $r_{\text{sub}.3.2}$ ” means “ r_3 , bit 2”, “ $r_{\text{sub}.2.1}$ ” means “ r_2 , bit 1”, and so on.

[0066] It can be shown according to Table 1, that an RNS representation of the value “ $99_{\text{sub}.10}$ ” under the said moduli-set $\{3_{\text{sub}.10}, 5_{\text{sub}.10}, 7_{\text{sub}.10}\}$ becomes “ $00110000_{\text{sub}.2}$ ” if we arrange residuals in such a way that a MSB of the RNS word is the MSB of the residual corresponding to the largest modulo $m_{\text{sub}.k}$ ($m_{\text{sub}.3}$ in this example), and the LSB of the RNS word is the LSB of the residual corresponding to the smallest modulo $m_{\text{sub}.1}$, and residuals are ordered between m_x and $m_{\text{sub}.1}$.

[0067] Consider a BNS with n bits, the BNS has a dynamic range of $2^{\text{sup}.n}$, and it can be shown that an RNS representation of an n -bit BNS always requires at least $n+1$ bits (combined bit-width of all residuals under the moduli-set) to represent a full dynamic range of $2^{\text{sup}.n}$ values.

[0068] It is known to all that it requires at least n bits to represent the $2^{\text{sup}.n}$ values $[0; 2^{\text{sup}.n}-1]$. Therefore, the only way to represent $2^{\text{sup}.n}$ RNS values in n bits is $M = 2^{\text{sup}.n}$. Because if $M > 2^{\text{sup}.n}$, it requires more than n bits to represent M . If $M = 2^{\text{sup}.n}$, then all factors of M will have to also be powers of 2, and if all factors of M are powers of two, then they all have 2 as a divisor themselves, so they are not co-prime. If they are not co-prime, then they are not a moduli set. If at least n bits are needed to represent an RNS representation and cannot be represented by n bits, then the RNS representation must have at least $n+1$ bits. For an 8-bit BNS value, the RNS representation will have at least 9 bits, for an 16-bit BNS value, the RNS representation will have at least 17 bits and so on.

[0069] This is inconvenient because an extra bit leads to additional data conversion between a main memory and an arithmetic logic unit (ALU), leading to bandwidth bottlenecks, reducing performance and increasing energy consumption at the same time. In addition, it requires more space to store RNS values than BNS values. For example, an 8-bit BNS value becomes a 9-bit RNS

value, so it occupies 2 pcs 8-bit positions in the memory or register-bank. It is also inconvenient to convert BNS constants such as FIR-filter coefficients, and neural network weights every time they are used, because such conversion requires energy.

2. Huffman Compression

[0070] The Huffman compression relies on a non-uniform distribution of the used values, so that frequently used values are encoded with “small” codes, and infrequently used values are encoded with “larger” codes. The most infrequently used codes will be larger than the original values, because the codes are so called “prefix codes”.

[0071] Currently, there are some researches on the application of the Huffman compression in an RNS, such as Huffman compression of RNS residuals; Huffman encoding of an entire RNS value, and specifically, RNS values is treated as a dataset to be compressed by the Huffman encoding, and each of the used RNS residual values will be assigned a Huffman “code”.

[0072] However, RNS values have high entropy, and Huffman compression works very bad on RNS values. Some RNS values will be encoded as Huffman codes larger than the original RNS values, and not all encoded values will have the same size.

[0073] Example: A sentence “this is an example of a Huffman tree” contains **36** characters of 16 different values (including the character SPACE or “”).

[0074] Since there are only 16 different values, we can encode the characters using 4 bits.

TABLE-US-00002 TABLE 2 4-bit Huffman Char code Occurrence code SPACE 0000 7 111 A 0001 4 010 E 0010 4 000 F 0011 3 1101 H 0100 2 1010 I 0101 2 1000 M 0110 2 0111 N 0111 2 0010 S 1000 2 1011 T 1001 2 0110 L 1010 1 11001 O 1011 1 00110 P 1100 1 10011 R 1101 1 11000 U 1110 1 00111 X 1111 1 10010

[0075] The above Table 2 shows characters, 4-bit codes, the number of occurrence in the sentence, and resulting Huffman codes. The most frequent letters (SPACE, A, E) all have 3-bit codes, while the most infrequent letters have 5-bit codes. The 5-bit codes are larger than the “straight” 4-bit codes, which could have been used to encode the data values directly. It can also be noted that 8 combinations (codes) can be formed from 3 bits, but only 3 combinations are used by the Huffman coding, 16 combinations can be formed from 4 bits, but only 7 combinations are used, and so on. This is due to the “prefix-code” requirement of the Huffman codes.

[0076] The above encoded sentence has a non-uniform distribution of values. The direct encoding as 4-bit values consumes $36 \times 4 = 144$ bits. The Huffman encoding consumes only 135 bits. Bit consumption is reduced by about 6%.

[0077] Consider a string of uniform distribution of values, where each of the 16 characters occur 2 times, for example, “AE FFXUXMNMHIST RORPLSTOPUNLHEIA”. There are $2 \times 16 = 32$ characters in total. Such a string needs $32 \times 4 = 128$ bits to be encoded directly, while the Huffman coding requires 134 bit. Bit consumption is increased by more than 5%.

[0078] It is commonly known that, Huffman coding requires the data to be non-uniformly distributed in order to be smaller than the source data. Huffman's agenda is to compress a natural language text. Most western languages have a non-uniform distribution of letters in a natural language, which makes Huffman compression feasible. General purpose data cannot be assumed to have such properties.

[0079] In general, $m_{\text{sub}.i}$ is far less than M , that is, $m_{\text{sub}.i} \ll M$, so a sequence of values of $r_{\text{sub}.i}$ will repeat itself

[00003] $\frac{M}{m_i}$

times through a range $[0: M-1]$ of the RNS, where $m_{\text{sub}.i}$ is one of the moduli in the moduli-set of the RNS, and $0 \leq r_{\text{sub}.i} < m_{\text{sub}.i}$. Values $[0; m_{\text{sub}.i}-1]$ will be close to uniformly represented in $r_{\text{sub}.i}$, and values $[m_{\text{sub}.i}; 2 \cdot \text{sup.p.sub.i}-1]$ will have a frequency of zero, where $p_{\text{sub}.i} = \lceil \log_{\text{sub}.2}(m_{\text{sub}.i}-1) \rceil$.

[0080] Even a non-uniform distribution of used values in the RNS may lead to a near-uniform distribution of values of residual $r_{\text{sub}.2}$, $0 \leq r_{\text{sub}.i} < m_{\text{sub}.i}$ under $m_{\text{sub}.i}$, where $m_{\text{sub}.i} \ll M$, for

all $m_{sub.i}$ in the moduli of the RNS.

[0081] An example is given here.

[0082] Consider the moduli $\{m_{sub.1}, m_{sub.2}\} = \{11_{sub.10}, 13_{sub.10}\}$, and

$M = m_{sub.1} \times m_{sub.2} = 143_{sub.10}$. Consider a dataset, BNS values $[0; 10_{sub.10}]$ occur $10_{sub.10}$ times each, BNS values $[11_{sub.10}; 100_{sub.10}]$ occur $5_{sub.10}$ times each, and BNS values $[101_{sub.10}; 142_{sub.10}]$ occur 1 time each. Such a dataset is highly non-uniform and the BNS would compress nicely with the Huffman compression. However, the occurrence of values of the residuals $r_{sub.1}$ and $r_{sub.2}$ are close to an ideal uniform distribution. In the case of $r_{sub.1}$, values 0, 1 occur $58_{sub.10}$ times each, and all other values $54_{sub.10}$ each. In the case of $r_{sub.2}$, values $[0; 9_{sub.10}]$ occur $48_{sub.10}$ times each, and other values $44_{sub.10}$, $39_{sub.10}$, and $39_{sub.10}$ times respectively. The residuals would not compress to a smaller result with the Huffman encoding.

[0083] It has been found that the Huffman compression of encoding the residuals is useless on RNS values. Furthermore, the Huffman compression does not hold if all values in the range $[0; 2^{sup.n}-1]$ are used, because some of the codes would be larger than the RNS word. Dynamic Huffman compression will be very expensive (time, area, energy, etc.), so static Huffman encoding will have to be used. This implies that the statistic distribution of the values of the processed data have to be known at the time of design of the chip. This is not the case for general purpose computing. Therefore this approach is useless in general purpose computing.

[0084] According to the above states, a representation in RNS of an n -bit BNS value requires at least $n+1$ bit, this leads to memory bandwidth issues and increased storage usage for above reasons. Repeated conversion of values (for example, software constant) from BNS to RNS requires much energy and reduces computation speed.

[0085] In addition, storage of intermediate values from an RNS requires: [0086] storage of 9-bit RNS format (which in praxis is 2×8 bits), which will lead to increased power consumption, memory bandwidth issues, and high use of intermediate storage area; and [0087] reverse conversion from RNS to BNS, storage as 8 bit, and forward conversion from BNS to RNS when the intermediate values are read again, which will lead to increased power consumption, and potential delays.

[0088] Given that, the present application propose a compact residual number system (CRNS) and a method for converting representations of values between RNS and CRNS. The CRNS representation is an alternative representation based on any usual RNS representation. CRNS uses only n bits to represent all $2^{sup.n}$ values of n -bit BNS.

[0089] RNS values, representing a BNS value x , where $2^{sup.n} \leq x < M$, cannot be represented by CRNS, but these RNS values aren't used and aren't needed in the present application.

[0090] The mapping between RNS representations and CRNS representations proposed by the present application is efficient in terms of area and energy, and is more efficient than regular BNS-to-RNS forward conversion and (Chinese Remainder Theorem) CRT-based or (Mixed-Radix System) MRS-based reverse conversion.

[0091] The mapping is based on a bijection between RNS representations and CRNS representations. The present application also proposes ways to optimize the bijection, in order to reduce area, power consumption and latency for the conversion logic.

[0092] The present application can be applied in computational elements, such as central processing unit (CPU), arithmetic logic unit (ALU), graphical processing unit (GPU), neural network (NN) acceleration units. In general, the solution proposed by the present application can be used in any product to which RNS is applied.

[0093] It seems that "AI on the edge" is already a big trend and is getting bigger in the near future. Here, inference processing takes place in mobile devices such as cellular phones, cameras, wearable devices and embedded devices in general. This implies that use of RNS technology is beneficial, and so is the concept of compact RNS (CRNS). So CRNS of the present application is a

number representation system, and it can use fewer bits to represent a BNS value compared to RNS. Specifically, CRNS can use n bits to represent values of an n -bit BNS, where the values are in a range of $[0; 2^{\sup.n-1}]$, and n is an integer.

[0094] The following describes the proposed solution of the present application in more details.

[0095] FIG. 1 is a schematic diagram of a method for converting first representations of values to second representations. A method (200) specifically includes the following steps:

[0096] Step 210: receive a first representation of a first value, where the first representation is a representation of the first value in an RNS, and the first representation contains $n+1$ bits.

[0097] Step 220: convert the first representation to a second representation according to a truncated bit, where the second representation is a representation of the first value in a CRNS, and the second representation contains n bits.

[0098] The truncated bit is predetermined, and it can be any one of the $n+1$ bits of the first representation.

[0099] The first value of is one of $2^{\sup.n}$ values that can be represented by an n -bit BNS, and a value range of the $2^{\sup.n}$ values is $[0, 2^{\sup.n-1}]$, n is an integer greater than or equal to 1.

[0100] In step 220, in the case that the truncated bit is equal to zero, the second representation is identical to remaining n bits of the first representation that exclude the truncated bit, and in the case that the truncated bit is equal to one, the second representation is identical to recoded remaining n bits of the first representation that exclude the truncated bit.

[0101] In other words, if the truncated bit is equal to zero, the second representation is a copy of remaining n bits of the first representation that exclude the truncated bit, and if the truncated bit is equal to one, the second representation is a copy of recoded remaining n bits of the first representation that exclude the truncated bit.

[0102] In the present application, the $2^{\sup.n}$ values correspond to $2^{\sup.n}$ first representations in the RNS, the $2^{\sup.n}$ values correspond to $2^{\sup.n}$ second representations in the CRNS, and the $2^{\sup.n}$ first representation and the $2^{\sup.n}$ second representation are in a bijection mapping relationship. In other words, any one of the $2^{\sup.n}$ values corresponds to a first representation in the RNS and corresponds to a second representation in the CRNS.

[0103] Method 200 is a general description about performing mapping from RNS representation(s) to CRNS representation(s). It should be understood that, mapping from CRNS representation(s) to RNS representation(s) should also be covered by the present application. Embodiments about mapping between CRNS and RNS will be introduced specifically later in the document.

[0104] Alternatively, “first representation” also can be replaced with “RNS representation”, and “second representation” also can be replaced with “CRNS representation”, and it is not restricted in the present application.

[0105] It should be understood that, an n -bit BNS can represent $2^{\sup.n}$ unique values x , where $0 \leq x < 2^{\sup.n}$. Consider a $(n+1)$ -bit RNS with a dynamic range M , where $2^{\sup.n} \leq M < 2^{\sup.n+1}$, the RNS can represent values x , where $0 \leq x < M$, that is $[0; M-1]$. In the present application, a bijection which maps between the $(n+1)$ -bit RNS representation and the n -bit CRNS representation is proposed. The bijection is a static bidirectional one-to-one mapping function. The bijection maps only $2^{\sup.n}$ RNS values (including zero) which correspond to n -bit BNS values in the range $0 \leq x < 2^{\sup.n}$. RNS values which correspond to BNS values larger than or equal to $2^{\sup.n}$ are disregarded in the present application, and may produce any output results from the bijection mapping function, including but not limited to zero, a constant value, a random value, or any other value.

[0106] Table 3 shows an example of mapping from BNS to RNS, and mapping from RNS back to BNS.

TABLE-US-00003 TABLE 3 A B C D E F X r.sub.2 r.sub.1 RNS BNS RNS BNS 0000 000 00
00000 0000 10000 BNS = 18 0001 001 01 00001 0111 10001 0100 0010 010 10 00010 1110 10010
1011 0011 011 00 00011 unused 10011 unused 0100 100 01 00100 1111 10100 1100 0101 101 10

00101 0001 10101 BNS = 19 0110 **110** 00 00110 1000 10110 0101 0111 000 01 001**11** unused
101**11** unused 1000 001 10 01000 1001 11000 0110 1001 010 00 01001 BNS = 16 11001 1101
1010 011 01 01010 0010 11010 BNS = 20 1011 **100** 10 010**11** unused 110**11** unused 1100 **101** 00
01100 0011 **11100** unused 1101 **110** 01 01101 1010 **11101** unused 1110 000 10 01110 BNS = 17
11110 unused 1111 001 00 011**11** unused **11111** unused

[0107] In table 3, the moduli-set is $\{3,7\}$, $m.sub.1=3$, $m.sub.2=7$, $M=m.sub.1 \times m.sub.2=21$, $n=4$, $2.sup.n=16$. The columns A and B show a mapping from BNS to RNS, and some of the RNS values have $MSB=1$, as shown in bold in column B. The columns C to F show a mapping from RNS back to BNS. The values marked “unused” in columns D and F correspond to $r.sub.1=3$ or $r.sub.2=7$, $r.sub.1$ is a residual under the modulo $m.sub.1$, and $r.sub.2$ is a residual under the modulo $m.sub.2$. “Unused” values can't happen, because $m.sub.1=3$ means $0 \leq r.sub.1 < 3$, and $m.sub.2=7$ means $0 \leq r.sub.2 < 7$. The cases of $r.sub.2=7$ represent binary combinations of the RNS-word, which are not valid RNS values, and hence also not valid BNS values. The present application is not concerned with these cases. The value marked “BNS=16” corresponds to BNS=16. This BNS value is outside the value range $[0; 2.sup.n-1]$. The values marked BNS=17, 18, 19 or 20 are similar. These RNS values are not used, because they represent a BNS value which can not be represented in the n -bit BNS.

[0108] As stated above, the bijection maps representations of $2.sup.n$ RNS values which corresponds to an n -bit BNS value in the range $0 \leq x < 2.sup.n$ to $2.sup.n$ representations in a CRNS respectively.

[0109] The n -bit second representation is created from remaining n bits by truncating a certain RNS-bit of the RNS word, that is, the $n+1$ bits of the first representation, and the “truncated bit” is called the bit $RNS.sub.j$ in the present application. In other words, a certain RNS bit is chosen from the first representation as the “truncated bit”. When converting a first representation in RNS of any one of the $2.sup.n$ values to a second representation in CRNS, if the bit $RNS.sub.j$ (that is, the truncated bit) is equal to “0”, the second representation is a copy of remaining n bits of the first representation that exclude the bit $RNS.sub.j$, that is, the n -bit second representation is identical to the remaining n bit of the first representation that excludes the bit $RNS.sub.j$, and if the bit $RNS.sub.j$ is equal to “1”, the second representation is obtained by performing recoding on the remaining n bits of the RNS word excluding the bit $RNS.sub.j$. Specifically, the remaining n bits of the RNS word excluding the bit $RNS.sub.j$ are recoded to a different, unused, CRNS value, that is, a CRNS value which is not occupied by the RNS values with the bit $RNS.sub.j=0$.

[0110] It can be shown that the number of RNS values which have $RNS.sub.j=1$ correspond exactly to the number of “unused” CRNS values. The “used” CRNS values correspond to the RNS values which have $RNS.sub.j=0$.

[0111] It has been proved by the present application, an n -bit BNS with $2.sup.n$ values $[0; 2.sup.n-1]$ leads to a $(n+1)$ -bit RNS. If h values of the $2.sup.n$ values have $RNS.sub.j=0$, then $h.sub.1=2.sup.n-h.sub.0$ values have $RNS.sub.j=1$. Then n -bit CRNS representation also has $2.sup.n$ possible values. When $c.sub.0$ CRNS values are mapped directly from RNS, then $c.sub.0=h.sub.0$. Then $c.sub.1=2.sup.n-c.sub.0$ values are left to be used for remapped RNS values which have $RNS.sub.j=1$. Since $c.sub.0=h.sub.0$, $c.sub.1=h.sub.1$, then the number of “unused” positions in the CRNS range will always correspond to the number of RNS values with $RNS.sub.j=1$. “Remapped” in the present application equals “recoded”, so “remapped values” also means “recoded values”.

[0112] An n -bit BNS can represent $2.sup.n$ values, the $2.sup.n$ values correspond to $2.sup.n$ first representations in the RNS and correspond to $2.sup.n$ second representations in the CRNS, and the $2.sup.n$ first representations and the $2.sup.n$ second representations are in a bijection mapping relationship.

[0113] The $2.sup.n$ first representations include f first representations of a first type and $(2.sup.n-f)$ first representations of a second type, the truncated bit of the first representation of the first type is

equal to one, and the truncated bit of the first representation of the second type is equal to zero.
[0114] If we consider a sequence of m ; integers $[0; m.\text{sub}.i-1]$, then the number of elements having $\text{MSB}=1$ is $h.\text{sub}.1$, and $h.\text{sub}.1$ can be calculated like $h.\text{sub}.1=m.\text{sub}.i-2.\text{sup}.p.\text{sup}.i.\text{sup}.-1$, $p.\text{sub}.i=\lceil \log.\text{sub}.2(m.\text{sub}.i-1) \rceil$. This sequence of numbers repeat $[00004]_{\frac{2^n}{m_i}}$ times though a value range $[0, 2.\text{sup}.n-1]$. The number $[00005]_{\frac{2^n}{m_i}}$

will not be an integer unless $m.\text{sub}.i$ is a power of 2. Therefore, there can be some values close to M , which also have $\text{MSB}=1$, but not the full amount of $h.\text{sub}.1$ above.

[0115] Let the value f be the number of values to be remapped, i.e., the number of RNS values, where $\text{RNS}.\text{sub}.j=1$. A general formula for f for any $m.\text{sub}.i$ is shown below. It is remarkable that f does not depend on neither the other modulo nor M , but only on n and $m.\text{sub}.i$, where n is the number of bit of a representation in the CRNS and BNS, $m.\text{sub}.i$ is the modulo to be truncated, and $p.\text{sub}.i$ is the number of bit required to represent $r.\text{sub}.i$ under $m.\text{sub}.i$, (i.e. $p.\text{sub}.i=\lceil \log.\text{sub}.2(m.\text{sub}.i-1) \rceil$).

[00006]

$$f = \begin{cases} \text{Math}.\frac{2^n}{m_i}.\text{Math}.(m_i - 2^{p_i-1}), & \text{if } \text{Math}.\frac{2^n}{m_i}.\text{Math}.\frac{m_i}{2^{p_i-1}} \leq 2^{p_i-1} \\ \text{Math}.\frac{2^n}{m_i}.\text{Math}.(m_i - 2^{p_i-1}) + \text{Math}.\frac{2^n}{m_i}.\text{Math}.\frac{m_i}{2^{p_i-1}}, & \text{if } \text{Math}.\frac{2^n}{m_i}.\text{Math}.\frac{m_i}{2^{p_i-1}} > 2^{p_i-1} \end{cases}$$

[0116] Example: let the moduli-set $m=\{m.\text{sub}.1, m.\text{sub}.2\}=\{5.\text{sub}.10, 13.\text{sub}.10\}$, $M=m.\text{sub}.1 \times m.\text{sub}.2=65.\text{sub}.10$, $n=6$, $p.\text{sub}.1=3$, $p.\text{sub}.2=4$, and the 7-bit moduli-set m has 65 values and can represent values in a value range of $[0; 63]$ of a 6-bit BNS.

[0117] Let's look at the values of x , $r.\text{sub}.1$, $r.\text{sub}.2$ near the end of the useful range $[0; 2.\text{sup}.n-1]=[0; 63]$.

TABLE-US-00004 TABLE 4 $x \quad r.\text{sub}.1 = |x|.\text{sub}.5 \quad r.\text{sub}.2 = |x|.\text{sub}.13$
55.sub.10 000 0011 56.sub.10 001 0100 57.sub.10 010 0101 58.sub.10 011 0110 59.sub.10 100 0111 60.sub.10 000 1000 61.sub.10 001 1001 62.sub.10 010 1010 63.sub.10 011 1011
 $2.\text{sup}.n = 64.\text{sub}.10$ 100 1100 $M = 65.\text{sub}.10$ 000 0000

[0118] As an example, we will first assume that $m.\text{sub}.1=5$ is the truncated modulo, also known as $r.\text{sub}.1$ is the truncated residual.

[0119] The final repetition of the sequence corresponding to $m.\text{sub}.1$ (we call it “ $m.\text{sub}.1$ sequence” in the following text) has only 4 of the 5 values, and all of these values have $\text{MSB}=0$.

$|2.\text{sup}.n|m.\text{sub}.1 \leq 2.\text{sup}.p.\text{sup}.i.\text{sup}.-1$. $\text{Math}.\frac{64}{5} \leq 2.\text{sup}.3-1$. $\text{Math}.\frac{64}{5} \leq 4$, true. Therefore, we are in the upper part of the equation of f above.

[00007] $\text{Math}.\frac{2^n}{m_1}.\text{Math}.$

is the number of complete repetitions of the $m.\text{sub}.1$ sequence.

[00008] $\text{Math}.\frac{2^n}{m_1}.\text{Math}.=\text{Math}.\frac{2^6}{5}.\text{Math}.=\text{Math}.\frac{12.8}{5}.\text{Math}.=12$.

$m.\text{sub}.i-2.\text{sup}.p.\text{sup}.i.\text{sup}.-1$ is the number of occurrences of $\text{MSB}=1$ in each repetition.

$m.\text{sub}.i-2.\text{sup}.p.\text{sup}.i.\text{sup}.-1=m.\text{sub}.1-2.\text{sup}.3-1=5-4=1$. The final repetition of the $m.\text{sub}.i$ sequence does not lead to any occurrence of $\text{MSB}=1$ in $r.\text{sub}.1$, so $f=12 \times 1=12$.

[0120] Now consider $m.\text{sub}.2$ is the truncated modulo. The last repetition of the sequence corresponding to $m.\text{sub}.2$ (we call it “ $m.\text{sub}.2$ sequence” in the following text) has only 12 of the 13 values present, and 4 of those have $\text{MSB}=1$.

[00009] $\text{Math}.\frac{2^n}{m_2}.\text{Math}.$

is the number of complete repetitions of the $m.\text{sub}.2$ sequence.

[00010] $\text{Math}.\frac{2^n}{m_2}.\text{Math}.=\text{Math}.\frac{2^6}{13}.\text{Math}.=\text{Math}.\frac{4.9231}{13}.\text{Math}.=4$.

$m.\text{sub}.2-2.\text{sup}.p.\text{sup}.2.\text{sup}.-1=m.\text{sub}.2-2.\text{sup}.4-1=13-8=5$. Therefore, there are $4 \times 5=20$

occurrences of $\text{MSB}=1$ in the first 4 repetitions of the $m.\text{sub}.2$ sequence. The final repetition of the $m.\text{sub}.2$ sequence has the first $|2.\text{sup}.n|m.\text{sub}.2=|2.\text{sup}.6|m.\text{sub}.2=|64|.sub.13=12$ of the 13 values in

the m.sub.2 sequence before 2.sup.n=64 is reached.

[0121] It requires p.sub.2 (p.sub.2=4) bits to represent r.sub.2, so the first 8 (2.sup.p.sub.2-1=2.sup.4-1=8) of these 12 values have MSB=0. The last 4 (|2.sup.n|m.sub.2-2.sup.p.sub.2-1=|64|.sub.13-2.sup.4-1=12-8=4) values of the m.sub.2 sequence have MSB=1, so f=4×5+4=24.

[0122] In addition, there are two special cases of f: [0123] For a modulo m.sub.i being the form m.sub.i=2.sup.q.sub.i+1:

$$[00011] f = \text{.Math.} \frac{2^n}{m_i} \text{.Math.}$$

... (minimum value of f as fraction of number of BNS values); [0124] For a modulo m.sub.i being the form m.sub.i=2.sup.q.sub.i:

$$[00012] f = \frac{2^n}{2} = 2^{n-1}$$

... (maximum value of f as fraction of number of BNS values).

[0125] The 23 smallest values of f for n=8 and m.sub.i≤128 are shown in graph of FIG. 2. Moduli of the form m.sub.i=2.sup.q.sub.i+1 are marked in black in FIG. 2. FIG. 2 shows an example of a number of recoded RNS values, f, in an 8-bit CRNS with corresponding 9-bit RNS and 9-bit BNS.

[0126] It can be shown according to FIG. 2, even though 9 is a fairly small number and a modulo of 9 will repeat many times

$$[00013] (\text{specifically, } \text{.Math.} \frac{2^8}{9} \text{.Math.} = 28 \text{times})$$

through the 8-bit value range of [0; 255]. There are only 5 values of moduli which result in smaller f and are not of the form m.sub.i=2.sup.q.sub.i+1. FIG. 3 shows an example of a trend graph of f for n=8. f continues to grow steadily towards

$$[00014] f = \frac{2^8}{2} = 128.$$

All moduli of the form m.sub.i=2.sup.q.sub.i have

$$[00015] f = \frac{2^8}{2} = 128.$$

Note that FIG. 3 is just for illustration of trend of f, and the value of f is not shown out.

[0127] Table 5 is an example of the method for converting representations of values from RNS to CRNS.

TABLE-US-00005 TABLE 5															
A	B	C	D	E	F	X	r.sub.2	r.sub.1	r.sub.2	r.sub.1	r.sub.2.1	r.sub.2.0			
r.sub.1.1	r.sub.1.0	G	0	0	0	000	00	0000	1	1	1	001	01	0101	2
2	2	2	010	10	1010	3	3	0	011	00	1100	4	4		
1	100	01	0011	5	5	2	101	10	0111	6	6	0	110	00	1001
7	0	1	000	01	0001	8	1	2	001	10	0110	9	2	0	010
00	1000	10	3	1	011	01	1101	11	4	2	100	10	1011	12	5
0	101	00	1110	13	6	1	110	01	1111	14	0	2	000		
10	0010	15	1	0	001	00	0100								

[0128] In table 5, m={3,7}, m.sub.1=3, m.sub.2=7, M=m.sub.1×m.sub.2=21, n=4, 2.sup.n=16. For an n-bit BNS, where n=4, only values belong to a values range [0; 15] are needed.

[0129] In table 5, as an example, the truncated bit is the MSB of r.sub.2. According to the method for converting an RNS representation to a CRNS representation proposed by the present application, if the truncated bit of a first representation is equal to zero, such as first representations of the values 0, 1, 2, 3, 7, 8, 9, and 10, the second representation is identical to remaining n bits of corresponding first representation that exclude the truncated bit, and the truncated bit of a first representation is equal to one, such as first representations of the values 4, 5, 6, 11, 12 and 13, the second representation is identical to remaining n bits of corresponding to first representation that exclude the truncated bit. For example, the first representation of 4 is “10001” which contain 5 bits, and the remaining 4 ((n+1)-1=5-1=n) bits of the first representation excluding the truncated bit is “0001”, and the second representation of 4 is identical to recoded remaining 4 bits, specifically, the remaining 4 bits are recoded as “0011”, and the second representation is identical to the recoded word, i.e., “0011”.

[0130] In the above embodiments, the truncated bit is selected from a residual, and we can call this residual the truncated residual. The truncated residual relates to a modulo, and we will call this modulo the truncated modulo, even if it is actually the residual, which is truncated. The n+1 bits of the first representation includes k parts, which are the k residuals, and the k residuals are in one-to-

one correspondence with the k moduli in the moduli-set. Alternatively, the k residuals are arranged in one of the following orders: [0131] in an ascending order according to corresponding values of the k moduli; [0132] in an descending order according to corresponding values of the k moduli; or in a random order.

[0133] For an example, in table 5, a first representation (i.e. RNS value) includes 5 bits, and the 5 bits include two parts. The said two parts are the two residuals $r_{\text{sub.1}}$ and $r_{\text{sub.2}}$, corresponding to the two moduli $m_{\text{sub.1}}$ and $m_{\text{sub.2}}$, respectively. The two parts are arranged in a descending order according to the values of the two moduli, specifically, in an order of $r_{\text{sub.2}}r_{\text{sub.1}}$. The truncated bit can be any of the bits contained in $r_{\text{sub.2}}$ or $r_{\text{sub.1}}$, where $r_{\text{sub.2}}$ contains 3 bits, and $r_{\text{sub.1}}$ contains 2 bits. As another example, the two parts are arranged in an ascending order like $r_{\text{sub.1}}r_{\text{sub.2}}$ according to values of the two moduli. As an example, if the moduli-set includes k moduli and k is larger than two, the k parts of the first representation can be arranged in a random order. For example, the moduli-set is $\{m_{\text{sub.1}}, m_{\text{sub.2}}, m_{\text{sub.3}}\} = \{3_{\text{sub.10}}, 5_{\text{sub.10}}, 7_{\text{sub.10}}\}$, where $k=3$, three parts of the first representation can be arranged as the order like $r_{\text{sub.1}}r_{\text{sub.2}}r_{\text{sub.3}}$, $r_{\text{sub.3}}r_{\text{sub.2}}r_{\text{sub.1}}$, $r_{\text{sub.2}}r_{\text{sub.1}}r_{\text{sub.3}}$ or $r_{\text{sub.2}}r_{\text{sub.3}}r_{\text{sub.1}}$ and so on, and it is not listed here one by one.

[0134] Alternatively, as an example, the $n+1$ bits of the first representation also can be arranged in a random order, and not be arranged to the k parts. That is to say, the $n+1$ bits of the k parts are interleaved, and the truncated bit is one of the interleaved $n+1$ bits.

[0135] Alternatively, the bijection mapping relationship between the $2^{\text{sup.}n}$ first representations and the $2^{\text{sup.}n}$ second representations is adjustable.

[0136] Note that it is possible to permute the output of the RNS-to-CRNS remapping function. That is to say, the actual CRNS values do not matter. The only thing which matters, is that the mapping is a bijection. The bijection property is a necessary and sufficient condition to ensure that the CRNS values can be mapped back to RNS values again. In order to do logic optimization, as an example, we will only permute the RNS values which are re-mapped (or “re-coded”), and we will not permute the RNS values which are “copied”. The $2^{\text{sup.}n}$ first representations include f first representations of a first type and $(2^{\text{sup.}n}-f)$ first representations of a second type. The truncated bit of the first representation of the first type is equal to zero, and the truncated bit of the first representation of the second type is equal to one. As an embodiment, it is possible to permute the bijection mapping relationship between the f first representations of the first type and corresponding f second representations. Take Table 5 as an example, the number of the first representation of the first type is six, and the six first representations of the first type correspond to values 4, 5, 6, 11, 12 and 13. The bijection mapping relationship shown in Table 5 is just an example, and the bijection mapping relationship of the six first representations of the first type and corresponding six second representation can be adjusted. In other words, the RNS values which have $\text{RNS}_{\text{sub.}j}=0$ in the RNS representation will not be permuted. The fact that the RNS values where $\text{RNS}_{\text{sub.}j}=1$ are re-coded and the RNS values where $\text{RNS}_{\text{sub.}j}=0$ are not, helps to clearly distinguish the present application from anything to do with the Huffman encoding.

[0137] It should be noted that, in the Huffman encoding, Huffman code will in general be different from the value it represents, any occurrence of the Huffman code being equal to the value it represents will be coincidental.

[0138] In other words, this embodiment is the permutation of outputs of a remapping function in such a way that any CRNS bit such as $\text{CRNS}_{\text{sub.}j}$ becomes a function of any one RNS bit such as $\text{RNS}_{\text{sub.}t}$ in logic expressions: $\text{CRNS}_{\text{sub.}j} = \text{RNS}_{\text{sub.}t}$ or $\text{CRNS}_{\text{sub.}j} = \text{not}(\text{RNS}_{\text{sub.}t})$ for any combination of j and t , and we will call this property “bit alignment”.

[0139] Alternatively, this embodiment also covers a situation where several combinations of RNS-bit and CRNS-bit are “bit alignment” at the same time, through a permutation of the remapped values.

[0140] According to embodiments stated above, n -bit CRNS representations are created by

truncating the bit RNS.sub.j of an RNS word. Alternatively, as an embodiment, the truncated bit is the MSB of an RNS word. Use of the MSB leads to fewer RNS values with RNS.sub.j=1, so less logic in a recoding function.

[0141] Let RNS.sub.j be the MSB of a modulo, m.sub.i, in the moduli-set. This modulo m.sub.i needs p.sub.i bits for its representation of the residual r.sub.i, where $p.sub.i = \lceil \log_2(m.sub.i - 1) \rceil$, $0 \leq r.sub.i < m.sub.i$. Of the m.sub.i possible values of r.sub.i, h.sub.0 = $2^{sup.p.sub.i} - 1$ values have MSB(r.sub.i)=0, h.sub.1 = m.sub.i - h.sub.0 values have MSB(r.sub.i)=1. If

m.sub.i = $2^{sup.p.sub.i}$, then h.sub.1 = h.sub.0 = $2^{sup.p.sub.i} - 1$. In all other circumstances, h.sub.1 < h.sub.0. Therefore, there can never be more values of r.sub.i where MSB(r.sub.i)=1 than MSB(r.sub.i)=0. If the modulo m.sub.i is not of the form m.sub.i = $2^{sup.p.sub.i}$, then there will be fewer values of r.sub.i where MSB(r.sub.i)=1 than where MSB(r.sub.i)=0.

[0142] Alternatively, as a further example, the truncated bit is the MSB of a modulo, and the modulo is being of the form m.sub.i = $2^{sup.p.sub.i} + 1$. It can be shown that use of the MSB of a modulo of the form m.sub.i = $2^{sup.p.sub.i} + 1$ leads to the fewest possible recoded RNS values, specifically, only one RNS value, within the sequence of m.sub.i residuals. Other forms of a modulo will have more than one recoded value per sequence of m.sub.i residuals.

[0143] It has been proved by the present application, if the modulo m.sub.i is of the form m.sub.i = $2^{sup.q.sub.i} + 1$, the values of the residual r.sub.i under m.sub.i will be in a range $0 \leq r.sub.i < m.sub.i$, then the highest value of r.sub.i is m.sub.i - 1 = $2^{sup.q.sub.i}$.

[0144] Therefore, only the value r.sub.i = m.sub.i - 1 = $2^{sup.q.sub.i}$ will have MSB(r.sub.i)=1. Any other form of modulo will lead to a higher fraction of values of having MSB(r.sub.i)=1.

[0145] Alternatively, as an example, the truncated bit of an RNS word is the MSB of a residual r.sub.k corresponding to a modulo m.sub.k of the form m.sub.k = $2^{sup.q.sub.k} + 1$, where the modulo m.sub.k is the highest value in the moduli-set.

[0146] It has been proved by the present application, if the modulo m.sub.i which has its MSB truncated is of the form m.sub.i = $2^{sup.q.sub.i} + 1$, then the number of values in the value range [0; $2^{sup.n} - 1$] which will have to be recoded is

$$[00016] f = \frac{2^n}{m_i}.$$

Since f and m.sub.i are inversely proportional, it is clear that the larger m.sub.i is, then smaller the f will be. If m.sub.i is the highest value modulo in the moduli-set, then no other modulo in the moduli-set will lead to a smaller value of f. Therefore, the use of the MSB of a residual corresponding to a modulo of the form m.sub.k = $2^{sup.q.sub.k} + 1$ which has the highest value in the moduli-set leads to the fewest possible recoded values in the bijection, compared with other choices of the truncated bit, for a given moduli-set.

[0147] Note that there may exist another modulo in another moduli-set than the said moduli-set, which leads to smaller value of f.

[0148] The method **400** for converting second representations to first representations are described below.

[0149] FIG. **4** is a schematic diagram of a method for converting the second representations of values to the first representations. A method (**400**) specifically includes the following steps:

[0150] Step **410**: receive a second representation of a first value, where the second representation is a representation of the first value in a CRNS, and the second representation contains n bits.

[0151] Step **420**: determine whether the second representation is identical in the CRNS and an RNS.

[0152] Step **430**: convert the second representation to a first representation, where the first representation is a representation of the first value in the RNS, and the first representation contains n+1 bits; [0153] where if the second representation is identical in the CRNS and the RNS, the first representation is identical to the n bits of the second representation and a truncated bit added by zero, and if the second representation is different in the CRNS and the RNS, the first representation is identical to recoded n bits of the second representation and the truncated bit added by one. In

other words, the first representation is identical to a combination of the n bits of the second representation and a truncated bit added by zero in the case that the second representation is identical in the CRNS and the RNS, or the first representation is identical to a combination of recoded n bits of the second representation and the truncated bit added by one in the case that the second representation is different in the CRNS and the RNS. The truncated bit in method **400** is predetermined.

[0154] In step **420**, if the second representation belongs to the f second representations corresponding to the f first representations of the first type, then the second representation is different in the CRNS and the RNS, and if the second representation does not belong to the f second representations corresponding to the f first representations, then the second representation is identical in the CRNS and the RNS, where the f second representations are predetermined. For f second representations corresponding to the f first representations of the first type, reference can be made to the embodiments of the method **200**, and it is not repeated here.

[0155] The first value is one of $2^{\text{sup}.n}$ values, the $2^{\text{sup}.n}$ values can be represented by an n -bit BNS, and a value range of the $2^{\text{sup}.n}$ values is $[0; 2^{\text{sup}.n}-1]$, n is an integer.

[0156] Note that converting CRNS representations to RNS representations is an inverse operation of converting RNS representations to CRNS representations. The truncated bit can be any one of the $n+1$ bits of the first representations in method **200**, so the truncated bit in method **400** can be added in any position of the n bits of the second representations, such as a position before the n bits, after the n bits, or between any two adjacent bits of the n bits. For the truncated bit in method **400**, reference can be made to explanations of the truncated bit in method **200**, and it is not repeated here.

[0157] Some detailed implementations are given below about how to converting $(n+1)$ -bit RNS representations to n -bit CRNS representations.

[0158] Alternatively, as an example, a 2:1 multiplexer and a logic function is used to implement the bijection from RNS to CRNS.

[0159] FIG. 5 is a schematic diagram of an embodiment of converting from RNS to CRNS. The notation " $n:0$ " means "bit n to bit 0", and the notation " $n-1:0$ " means "bit $n-1$ to bit 0". The notation " $n:0 \setminus j$ " means "bit n to 0 except bit j ", and the notation " $\setminus$ " is borrowed from math "set-theory", where " $\setminus$ " is the exclusion operator. These notations are applicable in the following embodiments, and will not be repeated.

[0160] As shown in FIG. 5, in case of the bit RNS.sub.j is equal to "0", the RNS representation is mapped directly to a CRNS representation, and the CRNS representation is identical to the remaining n bit of the RNS representation that excludes the bit RNS.sub.j . In case of the bit RNS.sub.j is equals to "1", the remaining n bits of the RNS representation that exclude the bit RNS.sub.j is recoded to a CRNS representation which is not occupied by the RNS representations with $\text{RNS.sub.j}=0$. In this embodiment, whether an RNS representation should be recoded depends on the bit RNS.sub.j of the RNS representation is equals to "0" or "1".

[0161] Alternatively, as an example, the truncated bit (that is, the bit RNS.sub.j) is the MSB of a modulo $m.\text{sub.i}$ of the form $m.\text{sub.i}=2^{\text{sup}.q.\text{sup}.i+1}$.

[0162] FIG. 6 is a schematic diagram of an embodiment of converting from RNS to CRNS. In this embodiment, the number of inputs to a re-coder function can be reduced by the number of bits, $p.\text{sub.i}=q.\text{sub.i}+1$, in the truncated modulo $m.\text{sub.i}$, so $h=n-p.\text{sub.i}$ in the FIG. 6. We know that $m.\text{sub.i}=2^{\text{sup}.q.\text{sup}.i+1}$ and the residual $r.\text{sub.i}$ under $m.\text{sub.i}$ belongs to the range $0 \leq r.\text{sub.i} < m.\text{sub.i}$, where $q.\text{sub.i}$ is an integer corresponding to $m.\text{sub.i}$. Therefore, the only occurrence of $\text{MSB}(r.\text{sub.i})=1$ is when $r.\text{sub.i}=m.\text{sub.i}-1=2^{\text{sup}.q.\text{sup}.i}$. Since a value of $r.\text{sub.i}$ is constant when the re-coder function is used, the value of $r.\text{sub.i}$ in the inputs of the re-coder function can be disregarded.

[0163] The number of inputs to the re-coder function is reduced by $p.\text{sub.i}$, also known as the number of bits of the residual $r.\text{sub.i}$, so complexity is reduced. Optional, as an example, the inputs

to the re-coder function is bit 0 to bit $h-1$, that is, $(h-1:0)$ as shown in FIG. 6. All other forms of truncated moduli lead to only one bit being removed from the inputs, namely the bit $RNS.sub.j$. In a moduli-set with k moduli, $m.sub.k$ is the modulo which has the largest value in the moduli-set. If the truncated modulo is $m.sub.k$, then advantages are maximized.

[0164] As shown in FIG. 6, it is assumed that the truncated bit is the MSB of the modulo which has the largest value in the moduli-set, and the modulo which has the largest value has form of $m.sub.k=2.sup.q.sup.k+1$, where $q.sub.k$ is an integer corresponding to $m.sub.k$. The modulo which has the largest value will be expressed as “the largest value modulo” instead in some embodiments for brevity. The truncated bit, that is the $RNS.sub.j$, in this embodiment shown in FIG. 6 is “bit n ”. The remaining n bits of the $(n+1)$ -bit RNS representation, excluding bit n , are bit $n-1$ to bit 0 specifically. In case of the bit n (an example of the bit $RNS.sub.j$) is equals to “0”, the $(n+1)$ -bit RNS representation will be mapped directly to an n -bit CRNS representation, and the n -bit CRNS representation is identical to the remaining n bits of the $(n+1)$ -bit RNS representation that exclude bit n . In case of the bit n of the $(n+1)$ -bit RNS representation is equals to “1”, the remaining n bits of the $(n+1)$ -bit RNS representation that exclude the truncated bit needs to be recoded. The recoded n bits is not occupied by the CRNS representations which corresponding to RNS representations with bit n is equals to “0”.

[0165] Note that no modulo will have more bits in its representation than the modulo which has the highest value, so if the modulo which has the highest value is the form $m.sub.k=2.sup.q.sup.k+1$, the advantages are maximized.

[0166] There may be other moduli $m.sub.t$ which have the same number of bits as the modulo $m.sub.k$, but no modulo will have more bits than the largest value modulo. In addition, any other moduli of the form $m.sub.t=2.sup.q.sup.t+1$ will have to have fewer bits than the largest modulo, because $q.sub.t$ will have to be smaller than $q.sub.k$, $q.sub.t < q.sub.k \Rightarrow p.sub.t < p.sub.k$.

[0167] This embodiment shown in FIG. 6 is a continuation of a foregoing embodiment, and the foregoing embodiment is the embodiment which use a modulo of the form $m.sub.i=2.sup.q.sup.i+1$ in order to reduce the number of recoded RNS values. The point of this embodiment is to use a 2:1 multiplexer and to reduce the number of inputs to the re-coder function at the same time as the number of recoded values are reduced in the foregoing embodiment. This embodiment shown in FIG. 6 is an improvement of the embodiment shown in FIG. 5 in the situation where the truncated modulo has a special form, that is $m.sub.i=2.sup.q.sup.i+1$.

[0168] Some detailed implementations are given below about how to converting n -bit CRNS representations back to $(n+1)$ -bit RNS representations.

[0169] FIG. 7 is a schematic diagram of an embodiment of converting from CRNS to RNS. A CRNS-to-RNS conversion uses a 2:1 multiplexer and a logic function (here called “mux-ctrl”) to implement the bijection from CRNS to RNS.

[0170] As shown in FIG. 7, a “mux-ctrl” function identifies all recoded CRNS representations, and determine whether an input CRNS representation is identical in the CRNS and the RNS or not. The 2:1 multiplexer passes CRNS representations which are identical in the CRNS and the RNS. A re-coder function recodes the CRNS representations which are different in the CRNS and the RNS. When the multiplexer passes CRNS representations directly to RNS, the output of the re-coder function is disregarded. This allows for optimization of implementation of a logic of the re-coder function. We know that, if the bit $RNS.sub.j=0$, the CRNS representations are passed unchanged, and if the bit $RNS.sub.j=1$, the re-coder function is used. Therefore, the bit $RNS.sub.j$ can be added as a constant before the multiplexer, as shown in FIG. 7. Specifically, the above $2.sup.n$ values of an n -bit BNS correspond $2.sup.n$ RNS representations and $2.sup.n$ CRNS representations, respectively. The $2.sup.n$ CRNS representations include f CRNS representations, and the f CRNS representations correspond f RNS representations of the first type in RNS. Each of the f CRNS representations is identical in the CRNS and the RNS. The f CRNS representations are predetermined and are known to the hardware, so the hardware can be designed to identify whether

a CRNS representation is identical in the CRNS and the RNS or not.

[0171] Alternatively, as an example, the truncated bit, that is, the bit $RNS.sub.j$, can also be inserted after the multiplexer, by using the output of the “mux-ctrl” function directly, as shown in FIG. 8.

FIG. 8 is a schematic diagram of an embodiment of converting from CRNS to RNS.

[0172] Alternatively, as another example, the present application also covers the case where the output of the “mux-ctrl” function is inverted, and the inverted value of the output is merged directly into the RNS value as the bit $RNS.sub.j$ after the multiplexer, as shown in FIG. 9. FIG. 9 is a schematic diagram of an embodiment of converting from CRNS to RNS. Note that the multiplexer inputs are swapped in FIG. 9, because a polarity of a control signal for the multiplexer is changed.

[0173] Alternatively, in an embodiment stated above, the truncated bit, that is the $RNS.sub.j$, is the MSB of a modulo of the form $m.sub.i=2.sup.q.sup.i+1$, then a width of the output of the re-coder function can be reduced by $p.sub.i=q.sub.i+1$ bit. In the case that the truncated bit is the MSB of the modulo of the form $m.sub.i=2.sup.q.sup.i+1$, then the width of the output of the re-coder function can be reduced by $p.sub.i=q.sub.i+1$, then the number of outputs of a logic function is reduced by $p.sub.i/n \times 100\%$, then the amount of gates and the occupied area are reduced by approximately the same relative amount, though not every output has the same logic function and complexity.

[0174] FIG. 10 is a schematic diagram of an embodiment of converting from CRNS to RNS. It shows that, $p=n-h+1$ bits of the truncated modulo $m.sub.i$ are located as the most significant bits, that is, bits $[n:h]$ of an RNS word. The bit $RNS.sub.j$ then becomes $RNS.sub.n$, actually, $j=n$. The present application should cover the case that the truncated residual $r.sub.i$ corresponding to the truncated modulo $m.sub.i$ can be anywhere in the RNS word, not only in the most significant bits. FIG. 10 will be overly complicated, if it is generalized in this way, so FIG. 10 is only an example.

[0175] FIG. 11 is a schematic block diagram of a chip or a chip system 10 of the present application.

[0176] As shown in FIG. 11, the chip (or the chip system) may include an interface 11 and a plurality of circuits 12. Alternatively, the interface 11 is configured to receive RNS representations of values, and transmit the RNS representations to the plurality of circuits 12. The plurality of circuits 12 perform the method 200, to convert the RNS representations to CRNS representations. Alternatively, the interface 11 is used to receive CRNS representations of values, and transmit the CRNS representations to the plurality of circuits 12. The plurality of circuits 12 perform the method 400, to convert the CRNS representations to RNS representations.

[0177] Alternatively, the interface 11 includes an input interface and an output interface, where the input interface is configured to receive RNS representations or second representations, and the output interface is configured to output the converting result of the plurality of circuits 12. As an embodiment, the interface 11 is an interface circuit.

[0178] As an embodiment, the function of the chip is implemented by hardware, such as the plurality of circuits 12, and the hardware includes one or more corresponding structures, such as multiplexers, controllers, re-coders and logical gates and so on.

[0179] In the several embodiments provided in this application, it should be understood that the disclosed system and method may be implemented in other manners. For example, the described apparatus embodiment is merely an example. For example, the unit division is merely logical function division and may be other division in actual implementation. For example, a plurality of units or components may be combined or integrated into another system, or some features may be ignored or not performed. In addition, the displayed or discussed mutual couplings or direct couplings or communication connections may be implemented through some interfaces. The indirect couplings or communication connections between the apparatuses or units may be implemented in electronic, mechanical, or other forms.

[0180] The units described as separate parts may be or may not be physically separate, and parts displayed as units may be or may not be physical units, may be located in one position, or may be distributed on a plurality of network units. Some or all of the units may be selected based on actual

requirements to achieve the objectives of the solutions of the embodiments.

[0181] In addition, functional units in the embodiments of this application may be integrated into one processing unit, or each of the units may exist alone physically, or two or more units are integrated into one unit.

[0182] The foregoing descriptions are merely specific implementations of this application, but are not intended to limit the protection scope of this application. Any variation or replacement readily figured out by a person skilled in the art within the technical scope disclosed in this application shall fall within the protection scope of this application. Therefore, the protection scope of this application shall be subject to the protection scope of the claims.

Claims

1. A method for converting presentations of values in different systems, performed by a chip, comprising: receiving a first representation of a first value, wherein the first representation is a representation of the first value in a residual number system (RNS), and the first representation contains $n+1$ bits; converting the first representation to a second representation according to a truncated bit, wherein the second representation is a representation of the first value in a compact residual number system (CRNS), and the second representation contains n bits; wherein the truncated bit is any one of the $n+1$ bits, and if the truncated bit of the $n+1$ bits is equal to zero, the second representation is identical to remaining n bits of the $n+1$ bits contained in the first representation that exclude the truncated bit, and if the truncated bit of the $n+1$ bits is equal to one, the second representation is identical to recoded remaining n bits of the $n+1$ bits contained in the first representation that exclude the truncated bit; wherein the first value is one of $2^{\text{sup}.n}$ values, the $2^{\text{sup}.n}$ values can be represented by an n -bit binary number system (BNS), and a value range of the $2^{\text{sup}.n}$ values is $[0; 2^{\text{sup}.n}-1]$, n is an integer.
2. The method according to claim 1, wherein the $2^{\text{sup}.n}$ values correspond to $2^{\text{sup}.n}$ first representations in the RNS, the 2^n values correspond to $2^{\text{sup}.n}$ second representations in the CRNS, and the $2^{\text{sup}.n}$ first representations and the $2^{\text{sup}.n}$ second representations are in a bijection mapping relationship.
3. The method according to claim 2, wherein the $2^{\text{sup}.n}$ first representations comprise f first representations of a first type and $(2^{\text{sup}.n}-f)$ first representations of a second type, the truncated bit of the first representation of the first type is equal to one, and the truncated bit of the first representation of the second type is equal to zero; wherein the f first representations of the first type correspond to f second representations, and each of the f second representations is identical to recoded remaining n bits of the corresponding first representation of the first type that exclude the truncated bit; wherein the $(2^{\text{sup}.n}-f)$ first representations of the second type correspond to $(2^{\text{sup}.n}-f)$ second representations, and each of the $(2^{\text{sup}.n}-f)$ second representations is identical to the remaining n bits of the corresponding first representation of the second type that exclude the truncated bit.
4. The method according to claim 3, the bijection mapping relationship between the f first representations of the first type and the f second representations is adjustable.
5. The method according to claim 1, wherein the $n+1$ bits of the first representation comprise k parts, the k parts are residuals corresponding to k moduli of the RNS, and the k parts are arranged in one of the following orders: in an ascending order according to corresponding values of the k moduli; in a descending order according to corresponding values of the k moduli; or in a random order.
6. The method according to claim 5, wherein the k parts comprise a first part, and the first part comprise $p_{\text{sub}.i}$ bits, the truncated bit is any one of the $p_{\text{sub}.i}$ bits, $p_{\text{sub}.i} = \lceil \log_{\text{sub}.2}(m_{\text{sub}.i}-1) \rceil$, $m_{\text{sub}.i}$ is a modulo of the k moduli corresponding to the first part, and the first part is any one of the k parts, $1 \leq i \leq k$, i and k are integers.

7. The method according to claim 6, wherein the truncated bit is a most significant bit (MSB) of the $p.\text{sub}.i$ bits.
8. The method according to claim 7, wherein the $m.\text{sub}.i$ is being the form $m.\text{sub}.i = 2^{\text{sup}.q.\text{sup}.i+1}$, $q.\text{sub}.i$ is an integer.
9. The method according to claim 8, wherein the $m.\text{sub}.i$ is the modulo that has the largest value of the k moduli.
10. A chip, comprising: an input interface, configured to receive a first representation of a first value, wherein the first representation is a representation of the first value in an RNS, and the first representation contains $n+1$ bits; a plurality of circuits, configured to: determine whether a truncated bit of the $n+1$ bits is equal to zero or one; convert the first representation to a second representation, wherein the second representation is a representation of the first value in a CRNS, and the second representation contains n bits; wherein the second representation is identical to remaining n bits of the $n+1$ bits of the first representation excluding the truncated bit in the case that the truncated bit is equal to zero, and the second representation is identical to recoded remaining bits of the $n+1$ bits of the first representation excluding the truncated bit in the case that the truncated bit is equal to one; and the first value is one of $2^{\text{sup}.n}$ values, the $2^{\text{sup}.n}$ values can be represented by an n -bit binary number system (BNS), and a value range of the $2^{\text{sup}.n}$ values is $[0; 2^{\text{sup}.n}-1]$, n is an integer.
11. The chip according to claim 10, wherein the $2^{\text{sup}.n}$ values correspond to $2^{\text{sup}.n}$ first representations in the RNS, the $2^{\text{sup}.n}$ values correspond to $2^{\text{sup}.n}$ second representations in the CRNS, and the $2^{\text{sup}.n}$ first representations and the $2^{\text{sup}.n}$ second representations are in a bijection mapping relationship.
12. The chip according to claim 11, wherein the $2^{\text{sup}.n}$ first representations comprise f first representations of a first type and $(2^{\text{sup}.n}-f)$ first representations of a second type, the truncated bit of the first representation of the first type is equal to one, and the truncated bit of the first representation of the second type is equal to zero; wherein the f first representations of the first type correspond to f second representations, and each of the f second representations is identical to recoded remaining n bits of the corresponding first representation of the first type that exclude the truncated bit; wherein the $(2^{\text{sup}.n}-f)$ first representations of the second type correspond to $(2^{\text{sup}.n}-f)$ second representations, and each of the $(2^{\text{sup}.n}-f)$ second representations is identical to the remaining n bits of the corresponding first representation of the second type that exclude the truncated bit.
13. The chip according to claim 12, the bijection mapping relationship between the f first representations of the first type and the f second representations is adjustable.
14. A chip, comprising: an input interface, configured to receive a second representation of a first value, wherein the second representation is a representation of the first value in a CRNS, and the second representation contains n bits; a plurality circuits, configured to: determine whether the second representation is identical in the CRNS and an RNS; and convert the second representation to a first representation, wherein the first representation is a representation of the first value in the RNS, and the first representation contains $n+1$ bits; wherein if the second representation is identical in the CRNS and the RNS, the first representation is identical to the n bits of the second representation and a truncated bit added by zero, and if the second representation is different in the CRNS and the RNS, the first representation is identical to the n bits of the second representation and the truncated bit added by one; wherein the first value is one of $2^{\text{sup}.n}$ values, the $2^{\text{sup}.n}$ values can be represented by an n -bit binary number system (BNS), and a value range of the $2^{\text{sup}.n}$ values is $[0; 2^{\text{sup}.n}-1]$, n is an integer.
15. The chip according to claim 14, wherein the $2^{\text{sup}.n}$ values correspond to $2^{\text{sup}.n}$ second representations in the CRNS, the $2n$ values correspond to $2^{\text{sup}.n}$ first representations in the RNS, and the $2^{\text{sup}.n}$ second representations and the $2^{\text{sup}.n}$ first representations are in a bijection mapping relationship.

16. The chip according to claim 15, wherein the $2^{\text{sup.}n}$ first representations comprise f first representations of a first type and $(2^{\text{sup.}n}-f)$ first representations of a second type, the truncated bit of the first representation of the first type is equal to one, and the truncated bit of the first representation of the second type is equal to zero; wherein the f first representations of the first type correspond to f second representations, and each of the f second representations is identical to recoded remaining n bits of the corresponding first representation of the first type that exclude the truncated bit; wherein the $(2^{\text{sup.}n}-f)$ first representations of the second type correspond to $(2^{\text{sup.}n}-f)$ second representations, and each of the $(2^{\text{sup.}n}-f)$ second representations is identical to the remaining n bits of the corresponding first representation of the second type that exclude the truncated bit.

17. The chip according to claim 16, wherein the f second representations corresponding to the f first representation of the first type are different in the CRNS and the RNS, and the $(2^{\text{sup.}n}-f)$ second representations corresponding to the $(2^{\text{sup.}n}-f)$ first representation of the second type are identical in the CRNS and the RNS; wherein the a plurality of circuits are configured to: determine whether the second representation is identical in the CRNS and the RNS by identifying whether the second representation belongs to the f second representations or not, wherein the f second representations are predetermined.
